

UNIVERSIDADE FEDERAL DE CAMPINA GRANDE
CENTRO DE ENGENHARIA ELÉTRICA E INFORMÁTICA

Residência em Microeletrônica – CI Expert

Verificação de Equivalência Formal

RTL $\times$ Netlist do Controlador de Vending Machine

Utilizando Synopsys Formality e Guidance via SVF

Relatório Técnico

Joelson Silva de Souza

Campina Grande – PB

Julho de 2026

Sumário

1	Introdução	2
2	Objetivos	2
3	Fundamentação Teórica	3
3.1	Síntese Lógica	3
3.2	Verificação Formal	3
3.3	Arquivo SVF	3
4	Metodologia	4
5	Configuração do Ambiente no Synopsys Formality	4
6	Utilização do fm_mk_script	6
7	Resultados	6
7.1	Guidance Summary	6
7.2	Operações registradas no arquivo SVF	7
7.3	Matching Results	8
7.4	Verification Results	8
8	Discussão dos Resultados	9
8.1	Propagação de constantes	10
8.2	Alteração de nomes	10
8.3	Operações de ungroup	10
8.4	Matching dos projetos	10
9	Análise Comparativa do Processo de Síntese	11
9.1	Resultado da verificação	12
9.2	Aviso FM-622	13
9.3	Undriven Nets	13
9.4	Synopsys Auto Setup	13
10	Conclusão	15
11	Referências	16

1 Introdução

O desenvolvimento de circuitos digitais modernos envolve uma sequência de etapas que transformam uma descrição comportamental do circuito em uma implementação física compatível com uma tecnologia de fabricação.

Inicialmente, o circuito é descrito utilizando uma linguagem de descrição de hardware (HDL), como Verilog ou SystemVerilog, em nível RTL (Register Transfer Level). Posteriormente, essa descrição é submetida ao processo de síntese lógica, no qual ferramentas especializadas convertem o código RTL em uma rede de portas lógicas pertencentes a uma biblioteca tecnológica.

Durante a síntese são aplicadas diversas otimizações automáticas visando reduzir área, potência e atraso crítico do circuito. Entre essas otimizações destacam-se propagação de constantes, remoção de lógica redundante, simplificação de expressões booleanas, alterações hierárquicas e mudanças de nomenclatura dos sinais internos.

Embora essas modificações alterem completamente a estrutura interna do circuito, a funcionalidade deve permanecer exatamente igual ao projeto RTL originalmente desenvolvido.

Para garantir essa equivalência utiliza-se a Verificação Formal de Equivalência (Logic Equivalence Checking – LEC), uma técnica baseada em métodos matemáticos que compara o projeto RTL com a netlist sintetizada sem depender de vetores de teste.

Neste trabalho foi realizada a verificação formal de equivalência do controlador de uma máquina de vendas (Vending Machine Controller), utilizando o Synopsys Design Compiler para síntese lógica e o Synopsys Formality X-2025.06-SP3 para comprovação da equivalência funcional entre o RTL e a netlist sintetizada.

Além disso, foi utilizado o arquivo SVF (Synopsys Verification Format), gerado automaticamente durante a síntese, permitindo ao Formality identificar corretamente as transformações realizadas pelo Design Compiler e acelerar o processo de comparação entre os dois projetos.

2 Objetivos

O objetivo principal deste trabalho é verificar formalmente que a implementação sintetizada apresenta exatamente o mesmo comportamento funcional do projeto RTL.

Os objetivos específicos são:

- realizar a síntese lógica do controlador utilizando o Synopsys Design Compiler;
- gerar a netlist sintetizada;
- gerar o arquivo SVF durante a síntese;
- configurar o ambiente de verificação utilizando o Synopsys Formality;
- carregar corretamente o projeto RTL e a netlist sintetizada;
- utilizar o arquivo SVF como guidance durante a verificação;
- executar o processo de matching;
- realizar a prova formal de equivalência;
- analisar todos os relatórios produzidos pela ferramenta.

3 Fundamentação Teórica

3.1 Síntese Lógica

A síntese lógica consiste na transformação automática de uma descrição RTL em uma implementação estrutural composta por células pertencentes a uma biblioteca tecnológica.

O Design Compiler interpreta o código HDL, realiza otimizações e gera uma netlist em Verilog composta por portas lógicas, flip-flops e demais células da biblioteca padrão.

Durante esse processo podem ocorrer:

- propagação de constantes;
- eliminação de lógica redundante;
- otimizações de área;
- otimizações de tempo;
- renomeação de sinais;
- alterações hierárquicas.

Como consequência, a estrutura interna do circuito sintetizado normalmente é bastante diferente da estrutura RTL original.

3.2 Verificação Formal

A Verificação Formal de Equivalência compara matematicamente dois circuitos digitais.

Neste trabalho foram utilizados dois modelos:

- Golden Design (Reference): descrição RTL em SystemVerilog;
- Revised Design (Implementation): netlist sintetizada pelo Design Compiler.

O objetivo da ferramenta é demonstrar que ambos produzem exatamente as mesmas saídas para qualquer combinação possível de entradas.

Ao contrário da simulação funcional, a verificação formal não depende da criação de casos de teste, permitindo analisar exaustivamente o comportamento lógico do circuito.

3.3 Arquivo SVF

Durante a síntese foi gerado automaticamente o arquivo SVF (Synopsys Verification Format).

Esse arquivo registra todas as principais transformações realizadas pelo Design Compiler, incluindo:

- mudança de nomes;

- propagação de constantes;
- alterações hierárquicas;
- informações da biblioteca utilizada;
- comandos de ambiente.

Ao carregar esse arquivo antes da execução do processo de matching, o Formality consegue reconhecer automaticamente as transformações realizadas durante a síntese, reduzindo o número de comparações necessárias e aumentando a robustez da verificação.

4 Metodologia

O fluxo utilizado durante esta atividade está representado na Figura 1.

RTL $\longrightarrow$ *DesignCompiler* $\longrightarrow$ *Netlist* $\longrightarrow$ *Formality* $\longrightarrow$ *Verification*

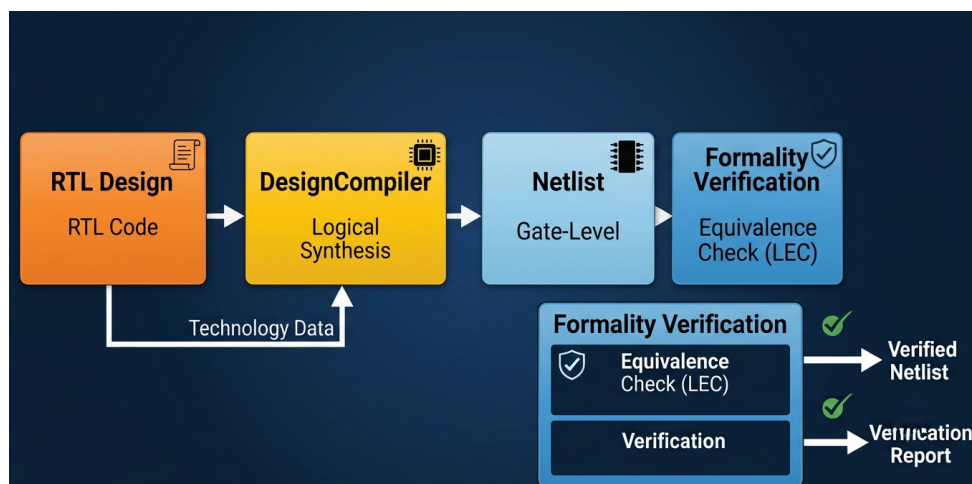


Figura 1: Fluxo utilizado durante a atividade.

5 Configuração do Ambiente no Synopsys Formality

Após a síntese lógica realizada no Synopsys Design Compiler, foi utilizada a ferramenta Synopsys Formality (versão X-2025.06-SP3) para verificar a equivalência funcional entre a descrição RTL e a netlist sintetizada.

O ambiente foi configurado por meio do script `formality.tcl`, responsável por automatizar todas as etapas da verificação.

O script realizou as seguintes operações:

- Carregamento da biblioteca tecnológica `saed32rvt_tt1p05v25c.db`;
- Definição do arquivo de guidance `default.svf`;

- Leitura dos arquivos RTL (vending_pkg.sv, credit_reg.sv, memory.sv, comparator.sv, subtractor.sv, control_unit.sv e vending_top.sv);
- Definição do projeto RTL como *Reference Design*;
- Leitura da netlist sintetizada vending_top_netlist.v;
- Definição da netlist como *Implementation Design*;
- Execução automática das etapas de match e verify;
- Geração dos relatórios finais de equivalência.

Após a execução do comando

```
source formality.tcl
```

todo o ambiente de verificação foi configurado automaticamente.

```
formality.tcl      formality.tcl      formality.tcl
formality.log      formality.tcl      formality.tcl
fm_shell (setup)> source formality.tcl
Loading db file '/home/joelson.souza/joelson_vending/libs/saad32rvt_ttlp05v25c.db'
SVF set to '/home/joelson.souza/joelson_vending/synth/reports/default.svf'.
Warning: 'read_verilog -09' will be obsoleted and removed from future releases
of Formality starting with I-2013.12 on all platforms. Please use 'read_sverilog -09'
instead. (FM-259)
No target library specified, default is WORK
Loading verilog file '/home/joelson.souza/joelson_vending/rtl/vending_pkg.sv'
Loading verilog file '/home/joelson.souza/joelson_vending/rtl/credit_reg.sv'
Loading verilog file '/home/joelson.souza/joelson_vending/rtl/memory.sv'
Warning: INITIAL statements are not supported. (File: /home/joelson.souza/joelson_vending/rtl/
memory.sv Line: 20) (FMR_VLOG-101)
Loading verilog file '/home/joelson.souza/joelson_vending/rtl/comparator.sv'
Loading verilog file '/home/joelson.souza/joelson_vending/rtl/subtractor.sv'
Loading verilog file '/home/joelson.souza/joelson_vending/rtl/control_unit.sv'
Loading verilog file '/home/joelson.souza/joelson_vending/rtl/vending_top.sv'
Current container set to 'r'
Setting top design to 'r:/WORK/vending_top'
Status: Elaborating design vending_top ...
Status: Elaborating design control_unit WIDTH=8 ...
Information: Created design named 'control_unit_WIDTH8'. (FE-LINK-13)
Status: Elaborating design credit_reg WIDTH=8 ...
Information: Created design named 'credit_reg_WIDTH8'. (FE-LINK-13)
Status: Elaborating design memory WIDTH=8 ...
Information: Created design named 'memory_WIDTH8'. (FE-LINK-13)
Status: Elaborating design comparator ...
Status: Elaborating design subtractor WIDTH=8 ...
Information: Created design named 'subtractor_WIDTH8'. (FE-LINK-13)
Status: Implementing inferred operators...

***** Library Checking Summary *****
Warning: 8 unlinked power cell(s) with unread pg pins.
Warning: 1 unlinked power cell(s) with unread save/restore retention signals.
Warning: 72 unlinked power cell(s) with no power down function on an ff or latch.
Use 'report_libraries -defects all' for more details.
*****

Top design successfully set to 'r:/WORK/vending_top'
Reference design set to 'r:/WORK/vending_top'
Warning: 'read_verilog -09' will be obsoleted and removed from future releases
of Formality starting with I-2013.12 on all platforms. Please use 'read_sverilog -09'
instead. (FM-259)
No target library specified, default is WORK
Loading verilog file '/home/joelson.souza/joelson_vending/synth/vending_top_netlist.v'
Current container set to 'i'
Setting top design to 'i:/WORK/vending_top'
Status: Implementing inferred operators...

***** Library Checking Summary *****
Warning: 8 unlinked power cell(s) with unread pg pins.
Warning: 1 unlinked power cell(s) with unread save/restore retention signals.
Warning: 72 unlinked power cell(s) with no power down function on an ff or latch.
Use 'report_libraries -defects all' for more details.
*****

Top design successfully set to 'i:/WORK/vending_top'
Implementation design set to 'i:/WORK/vending_top'
Reference design is 'r:/WORK/vending_top'
Implementation design is 'i:/WORK/vending_top'
Status: Checking designs...
```

Figura 2: Execução do script formality.tcl.

6 Utilização do fm_mk_script

Como etapa complementar da atividade foi utilizado o utilitário `fm_mk_script`.

Essa ferramenta utiliza o arquivo SVF produzido durante a síntese para gerar automaticamente um script inicial de configuração do Formality.

O comando utilizado foi

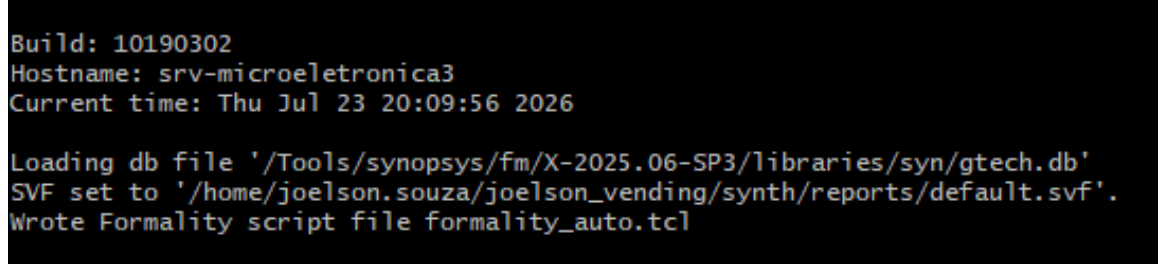
```
fm_mk_script ../synth/reports/default.svf \  
-o formality_auto.tcl
```

O script gerado foi comparado com o script desenvolvido manualmente.

As principais diferenças observadas foram:

- identificação correta da biblioteca tecnológica;
- identificação automática do arquivo SVF;
- utilização de caminhos absolutos;
- necessidade de ajuste manual na leitura dos arquivos RTL;
- ausência dos comandos finais de verificação (`match`, `verify` e geração dos relatórios).

Assim, o script manual foi mantido como fluxo definitivo da atividade.



```
Build: 10190302  
Hostname: srv-microeletronica3  
Current time: Thu Jul 23 20:09:56 2026  
  
Loading db file '/Tools/synopsys/fm/X-2025.06-SP3/libraries/syn/gtech.db'  
SVF set to '/home/joelson.souza/joelson_vending/synth/reports/default.svf'.  
Wrote Formality script file formality_auto.tcl
```

Figura 3: Geração automática do script utilizando o utilitário `fm_mk_script`.

7 Resultados

7.1 Guidance Summary

Durante o carregamento do arquivo `default.svf` o Formality interpretou todas as informações produzidas pelo Design Compiler.

O resumo apresentado pela ferramenta foi:

Comando	Accepted	Rejected	Pending	Unprocessed	Total
change_names	5	0	0	0	5
environment	4	0	0	0	4
reg_constant	48	0	0	0	48
ungroup	2	0	0	0	2
Total	59	0	0	0	59

Tabela 1: Resumo das operações lidas do arquivo SVF.

Todos os comandos presentes no arquivo SVF foram aceitos pela ferramenta.

Nenhuma operação foi rejeitada.

Isso demonstra que o arquivo de guidance gerado durante a síntese é compatível com o projeto carregado no Formality.

```

***** Guidance Summary *****
              Status
Command      Accepted  Rejected  Pending  Unprocessed  Total
-----
change_names :         5         0         0         0         5
environment  :         4         0         0         0         4
reg_constant :        48         0         0         0        48
ungroup      :         2         0         0         0         2
-----
Total        :        59         0         0         0        59

```

Figura 4: Resumo das operações do arquivo SVF.

7.2 Operações registradas no arquivo SVF

O relatório gerado mostrou que as otimizações realizadas durante a síntese foram corretamente reconhecidas pelo Formality.

As principais categorias de operações foram:

- 48 operações `guide_reg_constant`;
- 5 operações `guide_change_names`;
- 4 operações `guide_environment`;
- 2 operações `guide_ungroup`.

As operações `guide_reg_constant` indicam que diversos registradores presentes no módulo de memória foram identificados pelo Design Compiler como constantes lógicas.

Como consequência, esses registradores foram substituídos por valores fixos, reduzindo a quantidade de lógica necessária na implementação física sem alterar o comportamento funcional do circuito.

As operações `guide_change_names` registram alterações de nomenclatura realizadas durante a síntese.

As operações `guide_environment` armazenam informações sobre a biblioteca tecnológica e os comandos utilizados na síntese.

Já as operações `guide_ungroup` informam modificações hierárquicas aplicadas durante o processo de compilação.

7.3 Matching Results

Após o processamento do arquivo SVF foi executado o comando `match`.

O resumo produzido pelo Formality foi:

Resultado	Quantidade
Compare Points matched by name	41
Matched Primary Inputs	6
Unmatched Compare Points	0
Unmatched Primary Inputs	0

Tabela 2: Resumo do processo de matching.

É importante observar que o relatório de Matching apresenta 41 Compare Points e 6 Primary Inputs.

Somando esses elementos obtém-se

$$41 + 6 = 47$$

que corresponde exatamente aos 47 elementos listados no arquivo `formality_matched.rpt`.

Portanto, não existe divergência entre os dois relatórios.

```
***** Matching Results *****
41 Compare points matched by name
0 Compare points matched by signature analysis
0 Compare points matched by topology
6 Matched primary inputs, black-box outputs
0(0) Unmatched reference(implementation) compare points
0(0) Unmatched reference(implementation) primary inputs, black-box outputs
*****
```

Figura 5: Resultado da etapa de Matching.

7.4 Verification Results

Após o casamento dos pontos foi executado o comando

`verify`.

O resultado apresentado pelo Formality foi

Verification SUCCEEDED

A Tabela abaixo apresenta o resumo da prova formal.

Critério	Resultado
Passing Compare Points	41
Failing Compare Points	0
Unmatched Compare Points	0
Status	Verification SUCCEEDED

Tabela 3: Resultado final da verificação formal.

O resultado comprova que todos os 41 pontos de comparação analisados pelo Formality são funcionalmente equivalentes entre o projeto RTL e a netlist sintetizada.

```
***** Verification Results *****
Verification SUCCEEDED
  ATTENTION: synopsys_auto_setup mode was enabled.
             See Synopsys Auto Setup Summary for details.
  ATTENTION: Inconsistency detected in design data with SVF.
             See Design Consistency Check Summary for details.
-----
Reference design: r:/WORK/vending_top
Implementation design: i:/WORK/vending_top
41 Passing compare points
-----
Matched Compare Points   BBPin   Loop   BBNet   Cut   Port   DFF   LAT   TOTAL
-----
Passing (equivalent)      0      0      0      0    21    20     0    41
Failing (not equivalent)  0      0      0      0     0     0     0     0
*****
User can remove this screen by typing: 1038 MB
```

Figura 6: Resultado final da verificação formal.

8 Discussão dos Resultados

Os resultados obtidos durante a verificação formal demonstram que o fluxo de síntese e verificação foi executado corretamente, permitindo comprovar a equivalência funcional entre o projeto RTL e a netlist sintetizada.

Durante o carregamento dos projetos, o Formality identificou corretamente o projeto RTL como *Reference Design* e a netlist sintetizada como *Implementation Design*. Em seguida, o arquivo de guidance `default.svf` foi processado com sucesso, permitindo que as transformações realizadas durante a síntese fossem reconhecidas automaticamente.

A leitura do arquivo SVF apresentou o seguinte resumo:

- 48 comandos `guide_reg_constant`;
- 5 comandos `guide_change_names`;
- 4 comandos `guide_environment`;
- 2 comandos `guide_ungroup`;

- nenhuma operação rejeitada.

Esses resultados indicam que todas as informações produzidas pelo Design Compiler puderam ser utilizadas pelo Formality durante o processo de verificação.

8.1 Propagação de constantes

A maior parte das operações registradas no arquivo SVF correspondeu aos comandos `guide_reg_constant`.

Esses comandos indicam que diversos registradores presentes no módulo de memória foram identificados pelo sintetizador como assumindo valores constantes durante todo o funcionamento do circuito.

Como consequência, o Design Compiler substituiu esses registradores por níveis lógicos constantes, reduzindo a quantidade de flip-flops e de lógica combinacional necessária para implementar o circuito.

Essa otimização diminui a área ocupada e pode contribuir para a redução do consumo de potência, sem alterar o comportamento funcional do projeto.

8.2 Alteração de nomes

As cinco operações `guide_change_names` registram alterações de nomenclatura realizadas automaticamente durante a síntese.

Durante esse processo, diversos sinais internos recebem novos nomes para adequação ao formato da netlist.

Sem o arquivo SVF, essas alterações poderiam dificultar o casamento entre os elementos do projeto RTL e da implementação sintetizada.

8.3 Operações de ungroup

O arquivo SVF também registrou duas operações `guide_ungroup`.

Essas operações indicam alterações hierárquicas realizadas durante a síntese, nas quais determinados módulos tiveram sua estrutura reorganizada.

Mesmo após essas modificações, o Formality conseguiu identificar corretamente a correspondência entre os elementos dos dois projetos.

8.4 Matching dos projetos

Após a leitura do SVF foi executada a etapa de `match`.

O resumo apresentado pela ferramenta foi:

- 41 Compare Points matched by name;

- 6 Matched Primary Inputs;
- 0 Compare Points sem correspondência;
- 0 Primary Inputs sem correspondência.

É importante observar que existe uma diferença entre o número de elementos listados no relatório de matching e o número de compare points utilizados durante a prova formal.

O arquivo `formality_matched.rpt` apresenta um total de 47 elementos, pois inclui:

- 41 Compare Points;
- 6 portas primárias de entrada.

Já durante a etapa de verificação somente os 41 Compare Points são efetivamente comparados quanto à equivalência funcional.

Portanto, os valores observados nos dois relatórios são consistentes entre si.

9 Análise Comparativa do Processo de Síntese

Nesta etapa foi realizada uma análise complementar do impacto das opções utilizadas durante a síntese lógica no comportamento da verificação formal.

Inicialmente, a síntese do projeto foi realizada utilizando o comando:

```
compile_ultra -no_autoungroup
```

A opção `-no_autoungroup` impede que o Design Compiler realize a desestruturação automática da hierarquia dos módulos durante o processo de otimização. Dessa forma, a estrutura hierárquica original do projeto é preservada, facilitando a correspondência entre o projeto RTL e a netlist gerada.

Posteriormente, foi realizada uma nova síntese utilizando o comportamento padrão do comando:

```
compile_ultra
```

sem a restrição de impedir o agrupamento automático. Nessa configuração, o Design Compiler possui maior liberdade para modificar a hierarquia interna do circuito, podendo realizar operações como agrupamento, reorganização e otimização estrutural dos módulos.

Após a nova síntese, o processo de verificação formal foi executado novamente utilizando o Synopsys Formality.

O resultado obtido foi:

Verification SUCCEEDED

A ferramenta apresentou o seguinte resumo:

Critério	Resultado
SVF Commands Accepted	63
SVF Commands Rejected	0
Compare Points matched by name	41
Matched Primary Inputs	6
Passing Compare Points	41
Failing Compare Points	0
Status	Verification SUCCEEDED

Tabela 4: Resultado da verificação formal após alteração da síntese.

Mesmo com as alterações realizadas pelo Design Compiler durante a síntese, incluindo mudanças hierárquicas registradas pelos comandos `guide_ungroup`, o Formality conseguiu identificar corretamente a correspondência entre o projeto RTL e a implementação sintetizada.

Na nova execução do SVF foi observado o seguinte comportamento:

- aumento dos comandos de guidance aceitos de 59 para 63;
- aumento dos comandos `guide_ungroup` de 2 para 5;
- aumento dos comandos `guide_environment` de 4 para 5;
- manutenção dos 41 Compare Points equivalentes;
- ausência de pontos com falha.

A diferença observada demonstra que a opção de agrupamento automático altera a estrutura interna da netlist, porém não altera a funcionalidade do circuito.

O arquivo SVF foi fundamental para informar ao Formality as transformações realizadas pelo Design Compiler, permitindo que a ferramenta realizasse a prova formal mesmo após modificações estruturais mais agressivas.

Portanto, a retirada da opção `-no_autooungroup` não causou perda de equivalência funcional, confirmando que as otimizações realizadas pelo sintetizador preservaram o comportamento lógico do controlador de vending machine.

9.1 Resultado da verificação

Após o casamento dos elementos foi executado o comando

```
verify
```

O resultado apresentado foi

Verification SUCCEEDED

A ferramenta informou:

- 41 Passing Compare Points;
- 0 Failing Compare Points;
- 0 Unmatched Compare Points.

Esse resultado demonstra que todos os pontos analisados apresentaram exatamente o mesmo comportamento lógico tanto no projeto RTL quanto na netlist sintetizada.

9.2 Aviso FM-622

Durante a leitura do projeto foi emitido o aviso

FM-622

Inconsistency found between Formality Design Files
and Synthesis guide files information.

Esse aviso ocorreu porque algumas informações presentes no arquivo SVF não coincidem exatamente com os arquivos utilizados durante a leitura do projeto.

Entretanto, o próprio Formality conseguiu interpretar corretamente todas as operações presentes no arquivo SVF, conforme demonstrado pelo Guidance Summary, no qual todas as 59 operações foram aceitas.

Além disso, o resultado final da verificação foi **Verification SUCCEEDED**, indicando que esse aviso não comprometeu a prova formal.

9.3 Undriven Nets

Durante a etapa de consistência o Formality também informou a existência de alguns sinais classificados como *undriven nets*.

Esses sinais normalmente correspondem a registradores inicializados por constantes, sinais não utilizados ou estruturas que foram simplificadas durante a síntese.

Como a ferramenta operou utilizando o modo `synopsys_auto_setup`, esses sinais foram tratados automaticamente durante a verificação.

O resultado final comprova que tais avisos não produziram diferenças funcionais entre o RTL e a implementação sintetizada.

9.4 Synopsys Auto Setup

Durante a execução do comando `verify` foi utilizado automaticamente o modo

`synopsys_auto_setup`

Esse modo configura automaticamente diversos parâmetros internos da ferramenta, como o tratamento de sinais sem direcionamento, estruturas de teste e opções de interpretação do código RTL.

O próprio relatório informa que os resultados são válidos considerando essas configurações automáticas.

10 Conclusão

Neste trabalho foi realizada a verificação formal de equivalência entre a descrição RTL de um controlador de máquina de vendas e a respectiva netlist gerada pelo Synopsys Design Compiler.

O ambiente de verificação foi configurado utilizando o Synopsys Formality, empregando o arquivo SVF produzido durante a síntese para orientar o processo de comparação entre os dois projetos.

O Guidance Summary mostrou que todas as 59 operações registradas no arquivo SVF foram aceitas pela ferramenta, demonstrando total compatibilidade entre as informações produzidas durante a síntese e o ambiente de verificação.

Na etapa de matching foram identificados 41 Compare Points e 6 portas primárias, totalizando 47 elementos casados.

Durante a prova formal todos os 41 Compare Points foram aprovados, não sendo identificados pontos com falha nem pontos sem correspondência.

O resultado final

Verification SUCCEEDED

comprova que a implementação sintetizada preserva integralmente o comportamento funcional descrito no projeto RTL.

Assim, conclui-se que as otimizações realizadas pelo Design Compiler não introduziram alterações funcionais no circuito, validando a equivalência entre a descrição em SystemVerilog e a netlist sintetizada.

11 Referências

1. Synopsys. *Formality User Guide*. Version X-2025.06-SP3.
2. Synopsys. *Design Compiler User Guide*. Version X-2025.06-SP3.
3. Synopsys. *Synopsys Verification Format (SVF) Documentation*.
4. Weste, N.; Harris, D. *CMOS VLSI Design: A Circuits and Systems Perspective*. Pearson.